



南開大學
Nankai University

南开大学

网络空间安全学院

《数据安全》课程作业

基于 Paillier 算法的隐私信息获取实验

姓名：_____

学号：_____

专业：_____

指导教师：_____

2026 年 4 月 21 日

目录

一、 实验要求	1
二、 github 仓库	1
三、 实验原理	1
(一) 同态加密	1
(二) 半同态加密	2
(三) Paillier 算法	2
四、 实验过程	3
(一) 实验环境安装	3
1 安装 Python 环境	3
2 安装 phe 库	3
3 验证环境正确性	3
(二) 基于 Python 的 phe 库完成加法和标量乘法的验证	3
(三) 隐私信息获取	5
(四) 扩展实验探索	7
(五) 性能测试与结果分析	9
五、 实验心得与体会	11

一、实验要求

1. 基于 Paillier 算法实现隐私信息获取：从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获取了哪一个消息，同时客户端能完成获取消息的解密。
2. 扩展实验：客户端保存对称密钥 k ，服务器端存储 m 个用对称密钥 k 加密的密文，通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

本实验在实现时进一步明确了以下边界条件：

- 威胁模型为单服务器、半诚实模型，服务器按协议执行，但会尝试根据收到的信息推断客户端的目标下标。
- 不考虑恶意客户端伪造查询、重放攻击与拒绝服务等主动攻击。
- 演示环境使用本机 localhost TCP 通信，保证“客户端/服务器”两端分离，同时便于复现实验。

二、github 仓库

本次实验的代码、测试、结果数据以及报告源码均已整理在本地项目目录中，核心内容位于：

- 实验/lab1/pir_lab/：核心协议、编码、服务端、客户端实现
- 实验/lab1/scripts/：运行入口、AES 密钥初始化脚本、benchmark 脚本
- 实验/lab1/tests/：自动化测试
- 实验/lab1/report/：当前实验报告

若后续需要提交到 GitHub，只需将该目录整体推送到个人仓库，并将仓库地址补充在本节即可。当前版本已满足课程实验的代码与报告留档要求。

三、实验原理

（一）同态加密

同态加密（Homomorphic Encryption, HE）是一类允许在密文上直接计算，并在解密后得到与明文上对应计算结果一致的密码机制。其核心价值在于：数据无需先解密就能被第三方处理，从而在计算过程中仍然保持隐私。

从运算能力看，同态加密通常分为以下几类：

- **加法同态**：若加密函数 E 满足 $E(m_1) \times E(m_2) = E(m_1 + m_2)$ ，则说明该方案支持加法同态。
- **乘法同态**：若加密函数 E 满足 $E(m_1)^n = E(m_1 \times n)$ ，则说明该方案支持密文与明文常数之间的乘法同态。
- **全同态**：若在一定条件下同时支持加法与乘法组合运算，则可在密文上完成更复杂的电路计算。

本实验所使用的 Paillier 方案属于加法同态方案，并支持“密文乘明文常数”的运算，因此非常适合构造 one-hot 选择向量式的 PIR。

(二) 半同态加密

半同态加密 (Semi-Homomorphic Encryption) 是同态加密的一个子类, 只支持一种主要运算类型。本实验中的 Paillier 即属于半同态加密, 它支持:

- 密文与密文相乘, 对应明文加法;
- 密文与明文常数相乘, 对应明文标量乘法。

对两个明文 m_1, m_2 而言, 若对应密文分别为 $c_1 = E(m_1)$ 、 $c_2 = E(m_2)$, 则有:

$$D(c_1 \cdot c_2) = m_1 + m_2.$$

对密文 $E(m)$ 与常数 k 而言, 则有:

$$D(E(m)^k) = k \cdot m.$$

正是由于具备这两个性质, 服务器才能在不知道客户端目标下标的情况下, 直接对加密选择向量与消息集合进行聚合计算。

(三) Paillier 算法

Paillier 公钥密码体制最早由 Pascal Paillier 于 1999 年提出。其主要过程如下:

1. 密钥生成

- 选取两个大素数 p, q ;
- 计算 $n = pq$ 与 $\lambda = \text{lcm}(p-1, q-1)$;
- 选择合适的 $g \in \mathbb{Z}_{n^2}^*$;
- 公钥为 (n, g) , 私钥与 λ 相关。

2. 加密

- 对消息 $m \in \mathbb{Z}_n$, 随机选择 $r \in \mathbb{Z}_n^*$;
- 计算

$$c = g^m r^n \bmod n^2.$$

3. 解密

- 通过

$$m = \frac{L(c^\lambda \bmod n^2)}{L(g^\lambda \bmod n^2)} \bmod n$$

恢复消息。

4. 加法同态验证

- 若 $c_1 = E(m_1)$ 、 $c_2 = E(m_2)$, 则

$$c_1 c_2 \bmod n^2 = E(m_1 + m_2).$$

在本实验中, Paillier 的使用方式并不是直接加密所有消息, 而是用于加密客户端的选择向量, 让服务器在不知下标的情况下完成同态聚合。

四、 实验过程

(一) 实验环境安装

1 安装 Python 环境

本次实验在 Windows PowerShell 中完成，首先切换到项目目录并激活 `datasec` 虚拟环境，再确认 Python 运行环境可正常调用。实际实验中使用的是 Python 3.11.15，因此无需再次安装 Python，仅需要检查其版本并确认命令行可正常调用。

查看 Python 版本

```
1 python --version
```

2 安装 phe 库

Paillier 的核心功能依赖 `phe` 库实现，因此首先需要完成安装：

安装 phe 依赖

```
1 conda activate datasec
2 python -m pip install -r requirements.txt
```

实际安装后，`phe`、`pycryptodome` 与 `pytest` 均已可用。

3 验证环境正确性

为了确认环境安装成功，可进入 Python 解释器或直接执行导入验证：

环境验证命令

```
1 python -c "from phe import paillier; from Crypto.Cipher import AES; print('ok
  ')"
```

若输出 `ok`，说明依赖已安装正确，可以继续后续实验。

(二) 基于 Python 的 phe 库完成加法和标量乘法的验证

在正式实现隐私信息获取之前，先验证 `phe` 库是否确实满足课程中涉及的加法同态与标量乘法性质。测试代码如下：

同态运算验证代码

```
1 from phe import paillier
2 import time
3
4 print("默认私钥大小: ", paillier.DEFAULT_KEYSIZE)
5 public_key, private_key = paillier.generate_paillier_keypair()
6 message_list = [3.1415926, 100, -4.6e-12]
7
8 time_start_enc = time.time()
9 encrypted_message_list = [public_key.encrypt(m) for m in message_list]
10 time_end_enc = time.time()
11 print("加密耗时(s): ", time_end_enc - time_start_enc)
```

```
12
13 time_start_dec = time.time()
14 decrypted_message_list = [private_key.decrypt(c) for c in
    encrypted_message_list]
15 time_end_dec = time.time()
16 print("解密耗时(s): ", time_end_dec - time_start_dec)
17
18 a, b, c = encrypted_message_list
19 a_sum = a + 5
20 a_sub = a - 3
21 b_mul = b * 6
22 c_div = c / -10.0
23
24 print("a+5=", private_key.decrypt(a_sum))
25 print("a-3=", private_key.decrypt(a_sub))
26 print("b*6=", private_key.decrypt(b_mul))
27 print("c/-10.0=", private_key.decrypt(c_div))
28 print((private_key.decrypt(a) + private_key.decrypt(b)) == private_key.
    decrypt(a + b))
```

该测试表明：

- phe 可以正确完成 Paillier 加密与解密；
- 支持密文相加对应明文相加；
- 支持密文与明文常数的标量乘法；
- 不支持两个密文之间的直接乘法。

因此，该库能够满足本实验构造 PIR 方案的需要。

```
>> print((private_key.decrypt(a) + private_
_key.decrypt(a + b))
>> '@ | python -
默认私钥大小: 3072
加密耗时(s): 0.9293711185455322
解密耗时(s): 0.2804832458496094
a+5= 8.1415926
a-3= 0.14159260000000007
b*6= 600
c/-10.0= 4.6e-13
True
```

图 1: Paillier 同态性质验证结果

(三) 隐私信息获取

在基础实验中，服务器保存一个消息列表，客户端希望只取其中某一项，同时不让服务器知道请求的是哪一项。根据 Paillier 的性质，可以设计如下方案：

- 服务器端：保存消息列表 $data_list = \{m_1, m_2, \dots, m_n\}$ 。
- 客户端：
 - 设定目标下标 pos ;
 - 构造 one-hot 选择向量 $\{0, \dots, 1, \dots, 0\}$;
 - 对其逐位加密，得到 $\{E(0), \dots, E(1), \dots, E(0)\}$;
 - 将加密后的选择向量发送给服务器。
- 服务器端：
 - 计算
$$c = m_1 \cdot enc_list[1] + \dots + m_n \cdot enc_list[n];$$
 - 将聚合密文 c 返回给客户端。
- 客户端：解密聚合结果，恢复目标消息。

本次实验没有停留在单脚本模拟，而是实现成了真实的 TCP 客户端/服务器结构。服务端启动命令如下：

基础实验服务端启动命令

```
1 python scripts/run_server.py --mode basic --host 127.0.0.1 --port 9101 --count 16
```

客户端查询命令如下：

基础实验客户端查询命令

```
1 python scripts/run_client.py --mode basic --host 127.0.0.1 --port 9101 --dataset-size 16 --key-size 2048 --index 5
```

本次实际运行得到的服务端日志为：

基础实验服务端日志

```
1 [2026-04-21 08:34:13,869] server mode=basic host=127.0.0.1 port=9101
   dataset_size=16
2 [2026-04-21 08:34:13,869] dataset preview=["record-01|name=Alice|dept=AI|
   score=82",
3 "record-02|name=Bob|dept=Security|score=89", "record-03|name=Carol|dept=
   Networks|score=96"]
4 SERVER_READY mode=basic host=127.0.0.1 port=9101 dataset_size=16
5 [2026-04-21 08:34:44,696] received query: mode=basic, dataset_size=16,
   request_bytes=20925
```

客户端输出为:

基础实验客户端输出

```
1 {  
2   "mode": "basic",  
3   "index": 5,  
4   "dataset_size": 16,  
5   "key_size": 2048,  
6   "request_bytes": 20925,  
7   "response_bytes": 1364,  
8   "keygen_ms": 1658.04,  
9   "query_build_ms": 1599.79,  
10  "network_ms": 373.06,  
11  "decrypt_ms": 29.52,  
12  "plaintext": "record-06|name=Frank|dept=Security|score=85"  
13 }
```

可以看到, 服务端只能看到一个长度为 16 的加密查询及其请求大小, 无法从日志中直接恢复目标下标; 客户端最终正确得到目标明文。这说明基础版实验已经实现了“查询位置隐私可被保护、但目标消息仍可正确恢复”的要求。

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1> python script  
s/run_server.py --mode basic --host 127.0.0.1 --port 9101 --count 16  
[2026-04-21 08:34:13,869] server mode=basic host=127.0.0.1 port=9101 dataset_si  
ze=16  
[2026-04-21 08:34:13,869] dataset preview=["record-01|name=Alice|dept=AI|score=  
82", "record-02|name=Bob|dept=Security|score=89", "record-03|name=Carol|dept=Ne  
tworks|score=96"]  
SERVER_READY mode=basic host=127.0.0.1 port=9101 dataset_size=16  
□
```

图 2: 基础实验服务端启动并加载数据集

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1> python script  
s/run_server.py --mode basic --host 127.0.0.1 --port 9101 --count 16  
[2026-04-21 08:34:13,869] server mode=basic host=127.0.0.1 port=9101 dataset_si  
ze=16  
[2026-04-21 08:34:13,869] dataset preview=["record-01|name=Alice|dept=AI|score=  
82", "record-02|name=Bob|dept=Security|score=89", "record-03|name=Carol|dept=Ne  
tworks|score=96"]  
SERVER_READY mode=basic host=127.0.0.1 port=9101 dataset_size=16  
[2026-04-21 08:34:44,696] received query: mode=basic, dataset_size=16, request_  
bytes=20925  
□
```

图 3: 基础实验服务端接收加密查询


```

plaintext: record-06|name=Frank|dept=Security|score=85
python scripts/run_client.py --mode basic --host 127.0.0.1 --port 9101 --dataset-size 16 --key-size 2048 --index 5
Eric\Desktop\2026Spring\数据安全\实验\lab1>
{
  "mode": "basic",
  "index": 5,
  "dataset_size": 16,
  "key_size": 2048,
  "request_bytes": 20925,
  "response_bytes": 1364,
  "keygen_ms": 1658.0373000033433,
  "query_build_ms": 1599.790899999789,
  "network_ms": 373.06399999943096,
  "decrypt_ms": 29.5150000199303,
  "plaintext": "record-06|name=Frank|dept=Security|score=85"
}
> (datasetec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1>

```

图 4: 基础实验客户端恢复目标明文

(四) 扩展实验探索

在扩展实验中, 服务器不再直接保存明文消息, 而是保存经 AES-GCM 加密后的密文包; 客户端保存本地 AES 对称密钥。查询阶段仍然使用 Paillier 构造隐私选择向量, 服务器返回目标 AES 密文包, 客户端再本地解密恢复明文。

AES 密钥初始化命令如下:

AES 密钥初始化命令

```
1 python scripts/init_aes_key.py demo_aes.key
```

扩展实验服务端启动命令如下:

扩展实验服务端启动命令

```
1 python scripts/run_server.py --mode aes --host 127.0.0.1 --port 9102 --count 16 --key-file demo_aes.key
```

扩展实验客户端查询命令如下:

扩展实验客户端查询命令

```
1 python scripts/run_client.py --mode aes --host 127.0.0.1 --port 9102 --dataset-size 16 --key-size 2048 --index 5 --key-file demo_aes.key
```

本次实际运行中, AES 密钥初始化结果与服务端日志如下:

扩展实验服务端日志

```

1 AES key written to demo_aes.key
2 [2026-04-21 08:39:08,362] server mode=aes host=127.0.0.1 port=9102
   dataset_size=16
3 [2026-04-21 08:39:08,362] dataset preview=["record-01|name=Alice|dept=AI|
   score=82",
4 "record-02|name=Bob|dept=Security|score=89", "record-03|name=Carol|dept=
   Networks|score=96"]
5 SERVER_READY mode=aes host=127.0.0.1 port=9102 dataset_size=16

```

```
6 [2026-04-21 08:39:36,354] received query: mode=aes, dataset_size=16,  
   request_bytes=20922
```

客户端输出如下:

扩展实验客户端输出

```
1 {  
2   "mode": "aes",  
3   "index": 5,  
4   "dataset_size": 16,  
5   "key_size": 2048,  
6   "request_bytes": 20922,  
7   "response_bytes": 1372,  
8   "keygen_ms": 2020.23,  
9   "query_build_ms": 1605.26,  
10  "network_ms": 531.57,  
11  "decrypt_ms": 47.49,  
12  "plaintext": "record-06|name=Frank|dept=Security|score=85"  
13 }
```

这说明即使服务器端只保存 AES 密文, 客户端依然可以通过 PIR 正确恢复目标密文并解密得到明文。与基础版相比, 扩展版进一步实现了服务端静态存储密文, 而最终明文恢复仅在客户端本地完成。

```
• (datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1>  
python scripts/init_aes_key.py demo_aes.key  
AES key written to demo_aes.key  
❖ python scripts/run_server.py --mode aes --host 127.0.0.1 --port 91  
02 --count 16 --key-file demo_aes.key  
[2026-04-21 08:39:08,362] server mode=aes host=127.0.0.1 port=9102  
dataset_size=16  
[2026-04-21 08:39:08,362] dataset preview=["record-01|name=Alice|d  
ept=AI|score=82", "record-02|name=Bob|dept=Security|score=89", "re  
cord-03|name=Carol|dept=Networks|score=96"]  
SERVER_READY mode=aes host=127.0.0.1 port=9102 dataset_size=16  
[2026-04-21 08:39:36,354] received query: mode=aes, dataset_size=1  
6, request_bytes=20922  
□
```

图 5: 扩展实验 AES 密钥初始化与服务端处理查询

```

python scripts/run_client.py --mode aes --host 127.0.0.1 --port 9102 --dataset-size 16 --key-size 2048 --index 5 --key-file demo_aes.key
{
  "mode": "aes",
  "index": 5,
  "dataset_size": 16,
  "key_size": 2048,
  "request_bytes": 20922,
  "response_bytes": 1372,
  "keygen_ms": 2020.2279000004637,
  "query_build_ms": 1605.2629999976489,
  "network_ms": 531.5708000052837,
  "decrypt_ms": 47.486099996604025,
  "plaintext": "record-06|name=Frank|dept=Security|score=85"
}
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1>

```

图 6: 扩展实验客户端恢复目标明文

(五) 性能测试与结果分析

为了使本次报告达到优于往年基础版报告的完成度，本实验补充了系统化 benchmark，对消息规模、密钥长度以及基础版/扩展版的开销进行统计。运行命令如下：

benchmark 运行命令

```
python scripts/benchmark.py
```

消息规模测试结果如表 1 所示。

m	平均总耗时/ms	平均请求字节/B	平均响应字节/B	说明
8	1845.71	10811.33	1363.00	请求向量较短，整体耗时最低
16	2699.26	20920.67	1364.00	请求大小随消息规模继续增长
32	4678.05	41142.67	1364.00	查询构造与网络传输继续增加
64	8769.53	81588.00	1363.67	请求体积约为 $m = 8$ 时的 7.55 倍

表 1: 消息规模对性能与通信量的影响 (2048-bit, 基础实验)

密钥长度测试结果如表 2 所示。

密钥长度/bit	平均总耗时/ms	平均请求字节/B	平均响应字节/B	说明
1024	454.14	10748.33	747.33	运行最快，但安全裕度较低
2048	2590.74	20921.67	1363.67	兼顾安全性与可演示性
3072	9100.20	31090.67	1979.67	计算与通信开销显著上升

表 2: Paillier 密钥长度对性能与通信量的影响 (基础实验, $m = 16$)

基础版与扩展版的开销对比如表 3 所示。

模式	平均总耗时/ms	平均请求字节/B	平均响应字节/B	平均解密耗时/ms
basic	2622.82	20923.00	1363.33	27.37
aes	2418.77	20917.33	1372.00	28.13

表 3: 基础版与扩展版对比 ($m = 16$, 2048-bit)

实验结果说明:

- 当 m 从 8 增长到 64 时, 请求大小从 10811.33 B 增至 81588.00 B, 而响应始终约为 1364 B, 说明主要通信开销来自客户端发送的加密选择向量;
- 密钥长度增大后, 密钥生成、查询构造和同态聚合成本同步上升; 3072-bit 相比 2048-bit 的总耗时约提升到 3.5 倍, 通信量也明显变大;
- 本轮本地 benchmark 中 aes 的平均总耗时略低于 basic, 但两者处于同一数量级。这一差异更适合视为本机多轮测试波动, 而不是“AES 模式天然更快”; 更稳妥的结论是扩展模式增加了响应密文包大小, 并引入了客户端本地 AES-GCM 解密步骤。

此外, 本实验还通过 `pytest` 完成了自动化验证:

自动化测试结果

```
1 .....  
   [100%]  
2 5 passed in 1.67s
```

```
(datasec) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1>  
python -m pytest -q  
..... [100%]  
5 passed in 1.67s
```

图 7: 自动化测试全部通过

```

• (dataset) PS E:\Users\Eric\Desktop\2026Spring\数据安全\实验\lab1> Get-Content res
ults\benchmark_tables.md
# Benchmark Tables
## Dataset Scale
| count | key_size | avg_total_ms | avg_request_bytes | avg_response_bytes |
| --- | --- | --- | --- | --- |
| 8 | 2048 | 1845.71 | 10811.33 | 1363 |
| 16 | 2048 | 2699.26 | 20920.67 | 1364 |
| 32 | 2048 | 4678.05 | 41142.67 | 1364 |
| 64 | 2048 | 8769.53 | 81588 | 1363.67 |

## Key Size Scale
| count | key_size | avg_total_ms | avg_request_bytes | avg_response_bytes |
| --- | --- | --- | --- | --- |
| 16 | 1024 | 454.14 | 10748.33 | 747.33 |
| 16 | 2048 | 2590.74 | 20921.67 | 1363.67 |
| 16 | 3072 | 9100.2 | 31090.67 | 1979.67 |

## Mode Compare
| mode | count | key_size | avg_total_ms | avg_request_bytes | avg_response_bytes |
| --- | --- | --- | --- | --- | --- |
| basic | 16 | 2048 | 2622.82 | 20923 | 1363.33 |
| aes | 16 | 2048 | 2418.77 | 20917.33 | 1372 |

```

图 8: benchmark 结果汇总表

五、实验心得与体会

本次实验参考往年 Lab02 的整体作业结构，在此基础上完成了更完整的工程化实现。整体方法与课件中基于 Paillier 的 one-hot PIR 思路一致，但在实现层面进一步扩展为真实的客户端/服务器结构，并补充了协议序列化、自动化测试和 benchmark 分析，使整个实验更接近一个可复现的小型系统。

在实验过程中，我对 Paillier 半同态加密的理解更加清晰：它并不适合做任意运算，但非常适合用于构造 one-hot 查询向量和聚合结果，这一点恰好对应了隐私信息获取问题的需求。同时，扩展实验也让我进一步理解了“存储保护”和“访问模式保护”可以通过对称加密与同态加密的组合来共同实现。

总体来看，本次实验不仅完成了题目要求，也对同态加密的适用边界、实现约束以及工程化落地方式有了更直接的认识。本次报告中的日志、截图、测试和 benchmark 数据均来自本人在 2026 年 4 月 21 日的实际运行结果。若继续扩展，可以进一步尝试恶意安全模型、多服务器 PIR 或者更高效的通信压缩方案，以提升系统的实际应用价值。